FedMIM-LoRA: Orthogonal Fixed-Down-Projection LoRA with Masked Image Modeling for Communication- and Memory-Efficient Federated Vision Transformers

Anik Biswas
Department of Electrical and Electronic Engineering, BUET
Dhaka, Bangladesh
1906125@eee.buet.ac.bd

Abstract

Fine-tuning Vision Transformers inside a federated learning loop is constrained by three problems at once: edge devices cannot hold the activations a full ViT backward pass requires, the two-matrix structure of Low-Rank Adaptation (LoRA) does not average correctly under FedAvg, and differential-privacy noise interacts badly with that same two-matrix structure. We show that freezing an orthogonally-initialized down-projection matrix A_0 and sharing it across all clients removes the second and third problems by construction: the server-side update collapses to a linear average, so there is no aggregation error to correct, and DP noise enters only through the trainable up-projection matrix, so the usual noise-times-noise cross term never appears. The remaining risk with any frozen-A scheme is that a poorly conditioned A_0 starves the gradient signal reaching B; we show that an orthonormal A_0 has condition number 1, which keeps that signal undistorted regardless of how the matrix was drawn. To address the memory problem we combine this Fixed-A LoRA with 75% Masked Image Modeling patch dropping, cutting attention activation memory rather than parameter count. We refer to the combination as FedMIM-LoRA, instantiate it with ViT-B/16 on CIFAR-100 under Dirichlet non-IID partitioning ($\alpha = 0.1$) across 10 clients, and lay out the full experimental protocol needed to validate it: baselines, communication and VRAM accounting, DP robustness curves, and an initialization ablation. We report the protocol and the theory here; Section 5 states plainly which numbers are still pending a full training run.

1. Introduction

Federated learning (FL) lets several clients train a shared model without moving their data off-device [11]. That property is exactly what makes it attractive for adapting foundation vision models: a hospital network, a set of phones, or a fleet of factory sensors can each contribute local updates to a Vision Transformer (ViT) [6] without ever pooling raw images on a server. In practice, though, plugging a ViT into FedAvg exposes three problems that do not show up when a single machine trains the whole model.

The first is memory. Self-attention costs grow quadratically in sequence length, and a 224×224 image already gives ViT-Base 196 patch tokens before any batching happens. A full fine-tuning pass on that many tokens routinely exceeds the 16 GB a consumer or edge GPU offers.

The second is parameter-efficient fine-tuning (PEFT) itself. Low-Rank Adaptation (LoRA) [8] keeps the pretrained weight matrix W_0 frozen and learns only a low-rank update $\Delta W = BA$, which is normally a good trade: far fewer parameters to store and transmit. The trouble is that BA is a product, and FedAvg averages A and B independently before multiplying them back together. Averaging first and multiplying second is simply not the same operation as multiplying first and averaging second. The two only agree when every client's $B_i A_i$ happens to be identical, which never holds under real data heterogeneity. The gap between them, which we and others call aggregation discordance or server-side aggregation bias, grows with how skewed each client's local class distribution is [2].

The third problem only appears once differential privacy enters the picture. DP-SGD clips and perturbs gradients with calibrated Gaussian noise [1]. If both LoRA matrices are trained and both are perturbed, the noisy update expands to a sum of four terms, one of which is a product of two independent noise matrices. That term is not linear in the privacy noise; it is quadratic, and it can dominate the useful signal well before the noise on either matrix alone would be a problem [13].

A natural fix for the second and third problems is to freeze one of the two LoRA matrices. Sun *et al.*'s FFA-LoRA does exactly this: it fixes the randomly-initialized A

and only trains B [13]. Averaging a linear quantity (B) is now exact, and the DP noise term is linear too, since a frozen matrix receives no noise. But freezing a *random* Gaussian A has a cost that shows up in practice. An arbitrarily conditioned A can shrink gradient signal along some directions and amplify it along others, which slows convergence and caps final accuracy relative to full LoRA [2, 5].

This paper argues that the fix for FFA-LoRA's accuracy gap is not to unfreeze A. Every method that does so, whether LoRA-FAIR's server-side correction [2], alternating-freeze schemes [4], or SVD-based reconstruction [15], reopens some version of the aggregation or noise problem it was trying to close. Instead, the fix is to choose *how* A is frozen. If the shared, frozen A_0 is initialized to have orthonormal rows rather than i.i.d. Gaussian entries, its condition number is exactly 1, and the backward pass through it neither shrinks nor amplifies the gradient reaching B (Section 4). This single change, Fixed-A LoRA with an orthogonal A_0 rather than a random one, keeps every aggregation and privacy guarantee that motivated freezing A in the first place while removing the specific mechanism that made frozen-random-A under-perform.

We combine this Orthogonal Fixed-A LoRA with Masked Image Modeling (MIM) [7], dropping 75% of input patches before they reach the transformer encoder, to address the memory problem on a separate axis. Masking shrinks the sequence length that attention has to process; it is an activation-memory fix rather than a parameter-count fix, and it is complementary to anything LoRA does. We call the combination **FedMIM-LoRA**. Concretely, this paper contributes:

- A derivation of the aggregation-discordance and DP noise-amplification terms for standard federated LoRA, and a proof that a globally shared, frozen A_0 eliminates both exactly, independent of how A_0 was initialized (Sections 3–4.2).
- A proof that orthonormal initialization of A_0 is what makes Fixed-A LoRA competitive: it gives A_0 a condition number of 1, so the client-to-server gradient signal through A_0 is preserved exactly rather than distorted (Section 4.5).
- An instantiation, FedMIM-LoRA, that pairs this initialization scheme with 75% MIM patch masking on ViT-B/16, targeting communication payloads and peak VRAM an order of magnitude below full fine-tuning (Section 4.7).
- A full experimental protocol, covering baselines, a CIFAR-100 Dirichlet non-IID setup, communication/VRAM accounting, DP-robustness curves, and an initialization ablation, specified precisely enough to reproduce (Section 5). We report which of these numbers are measured and which are still outstanding.

2. Related Work

2.1. Federated LoRA and the Aggregation Problem

FedIT-style methods apply FedAvg [11] directly to both LoRA matrices, which is simple but inherits the aggregation discordance we derive in Section 3. FFA-LoRA [13] was the first to sidestep this by freezing the down-projection matrix across all clients and training only the up-projection matrix, which turns the federated update into an exact linear average and halves communication cost. The trade-off is expressivity. Because A is frozen at a random Gaussian draw, the update is confined to whatever subspace that draw happens to span, and several follow-ups report an accuracy gap relative to full LoRA under heterogeneous data [2, 4].

Most subsequent work tries to recover that expressivity without giving back the aggregation guarantee outright. LoRA-FAIR [2] lets both matrices train locally but adds a server-side correction term to the up-projection matrix after averaging, keeping the corrected module close to the naive average so shared information is not lost. RoLoRA [4] alternates which matrix is frozen across communication rounds and motivates the choice with a multitask linear-representation-learning argument for why both projections need to be learned eventually. Ravan [12] reparameterizes the update as a sum of multiple low-rank heads and trains only a lightweight core matrix and scale per head, which targets computational heterogeneity across clients rather than aggregation bias directly. Fed-HeLLo [16] instead allocates which layers get trainable LoRA modules per client, based on local resource budgets and layer importance. Fed-Momentum [15] aggregates correctly first and then applies SVD to the aggregated update to redistribute it back into a fixed-rank module, aiming to prevent the loss of directional information ("training momentum") that a naive low-rank projection would otherwise discard.

All of these methods recover expressivity by reintroducing some server-side computation, extra communication, or a scheduling decision. FedMIM-LoRA takes the opposite route: keep A frozen and shared, exactly as FFA-LoRA does, and address the expressivity gap purely through how A is initialized.

2.2. Orthogonal and Structured Initialization

A separate line of work argues that LoRA's sensitivity to initialization is itself the deeper issue, independent of the federated setting. HRP [5] shows, via a gradient-flow analysis of both symmetric and asymmetric (one-matrix-frozen) LoRA, that random initialization does not converge to the best available low-rank approximation, and proposes briefly training at a higher rank to estimate a better initial direction before dropping back to the target rank. SOS-LoRA [3] parameterizes the update as a sum of static, orthogonally-initialized low-rank experts with a fixed multi-scale scaling

scheme, and reports that convergence becomes independent of the initialization's condition number once orthogonality is enforced. Our Section 4.5 gives a direct instance of this same principle for the federated, frozen-A setting: an orthonormal A_0 has condition number 1, so it neither shrinks nor inflates the gradient signal reaching B, which is precisely the failure mode that limits vanilla FFA-LoRA.

2.3. Differential Privacy in Federated LoRA

Differential privacy compounds the aggregation problem rather than replacing it. When DP-SGD noise is added independently to both LoRA matrices, the product $\tilde{B}\tilde{A}$ contains a noise-times-noise cross term that scales worse than either matrix's noise alone (Section 4.4). That is a second, independent reason to avoid training both matrices under DP, alongside aggregation discordance. FedASK [14] addresses this with a two-stage sketching pipeline: clients send privacy-preserving sketches of their local updates, and the server reconstructs and refactorizes the global matrices via SVD, allowing both adapters to update without incurring the quadratic penalty. FedSVD [10] takes a related approach in which clients optimize only the up-projection matrix locally while the server periodically refactorizes the aggregated product via SVD to produce a new, adaptively-updated down-projection matrix, letting the frozen-side matrix track principal directions over time without ever exposing it to local DP noise. Both methods are more expressive than a permanently frozen A_0, at the cost of an SVD on the server every round. FedMIM-LoRA instead removes the quadratic noise term structurally, by never training A at all, and asks orthogonal initialization alone to close the resulting expressivity gap: a design point that neither FedASK nor FedSVD occupies.

2.4. Memory-Efficient Vision Transformers

Separately from the aggregation and privacy literature, Masked Autoencoders [7] showed that a ViT encoder can process a small, random subset of image patches and still learn strong representations, since the excluded patches are only needed by a lightweight decoder for the pretraining objective. We reuse the same masking mechanism at fine-tuning time, purely as a compute- and memory-reduction device. Dropping 75% of patches before the encoder shrinks the token sequence from 196 to 49, which cuts the quadratic cost of self-attention by roughly $16\times$ and is what makes ViT-Base fine-tuning fit on edge-class GPUs at all. This addresses activation memory, a dimension that Fixed-A LoRA, which only reduces trainable *parameter* count, does not touch on its own.

3. Bottlenecks in Federated ViT Fine-Tuning

Before presenting FedMIM-LoRA, it is worth being precise about what actually breaks when a ViT is fine-tuned inside FedAvg, since the three bottlenecks below have different root causes and, as we show in Section 4, different fixes.

Activation memory. The memory a training step needs is not just a function of parameter count; it is dominated by the intermediate activations kept around for the backward pass. Freezing a layer removes the need to store *dynamic* activations for that layer's linear functions, but does not remove *static* activations produced by the non-linearities (layer norms, GELU, softmax) that sit between trainable modules. Those still have to be stored so gradients can flow to whichever layers remain trainable [7]. For a ViT-Base/16 model at 224×224 resolution, this means 196 patch tokens flowing through every attention block, and since self-attention cost scales as $O(N^2)$ in the number of tokens N, that is already enough to push a full or LoRA fine-tuning pass past 16 GB of VRAM on typical edge hardware. Freezing parameters (as LoRA already does) helps, but only partially; it does not touch the token-count term.

Aggregation discordance. FedAvg was designed for parameters that combine linearly: averaging weights on the server and averaging weights after training locally are meant to agree, at least approximately [11]. LoRA's update $\Delta W = BA$ is a *product* of two matrices, and a product does not commute with averaging. Averaging A and B separately across clients and then multiplying the averages gives a different matrix than multiplying each client's B_iA_i locally and then averaging those products. We derive the exact size of this gap in Section 4.2; qualitatively, it grows with how different clients' local updates are from one another, which is precisely what non-IID data partitioning produces.

DP noise amplification. When differential privacy is required, DP-SGD clips per-example gradients and adds calibrated Gaussian noise before any update leaves the client [1]. If both LoRA matrices are perturbed independently, multiplying the two noisy matrices back together produces a cross term that is quadratic in the noise magnitude, rather than linear, which can overwhelm the fine-tuning signal well before a linear noise term would [13]. This is a separate failure mode from aggregation discordance: it would appear even with IID data, but it has the same underlying cause. LoRA's update is a product, and products do not treat noise the way sums do.

Section 4 shows that a single design choice, a globally shared and permanently frozen down-projection matrix, removes the second and third bottlenecks exactly, and that choosing that frozen matrix to be orthogonally initialized is what keeps the first fix from costing accuracy. Section 4.7 then addresses the memory bottleneck separately, through input-side patch masking.

4. FedMIM-LoRA

4.1. Preliminaries

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA parameterizes the fine-tuning update as $\Delta W = BA$, with down-projection $A \in \mathbb{R}^{r\times k}$, up-projection $B \in \mathbb{R}^{d\times r}$, and rank $r \ll \min(d,k)$ [8]. In a federation of N clients, client i produces local matrices A_i and B_i after local training.

4.2. Aggregation Discordance

The federated objective is to reach the same result as if a single model had been trained on the union of all clients' data. For LoRA, the corresponding *ideal* update averages the local products directly:

$$\Delta W_{\text{Ideal}} = \frac{1}{N}\sum_{i=1}^{N} \Delta W_i = \frac{1}{N}\sum_{i=1}^{N} B_i A_i. \tag{1}$$

FedAvg, however, communicates A_i and B_i separately and averages each on the server to conserve bandwidth:

$$\bar{B} = \frac{1}{N}\sum_{i=1}^{N} B_i, \qquad \bar{A} = \frac{1}{N}\sum_{i=1}^{N} A_i. \tag{2}$$

The global update actually applied is $\Delta W_{\text{FedAvg}} = \bar{B}\bar{A}$. Expanding the product exposes the problem:

$$\Delta W_{\text{FedAvg}} = \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right)\left(\frac{1}{N}\sum_{j=1}^{N} A_j\right) = \frac{1}{N^2}\sum_{i=1}^{N}\sum_{j=1}^{N} B_i A_j. \tag{3}$$

Splitting the double sum into matched ($i = j$) and cross ($i \neq j$) terms gives

$$\Delta W_{\text{FedAvg}} = \frac{1}{N^2}\sum_{i=1}^{N} B_i A_i \;+\; \frac{1}{N^2}\sum_{i\neq j} B_i A_j, \tag{4}$$

so the discordance, the exact gap between what FedAvg computes and what the federation intended, is

$$\mathcal{E} \;=\; \Delta W_{\text{FedAvg}} - \Delta W_{\text{Ideal}} \;=\; \frac{1}{N^2}\sum_{i\neq j} B_i A_j \;-\; \frac{N-1}{N}\,\Delta W_{\text{Ideal}}. \tag{5}$$

The surviving term is a sum of $N(N-1)$ *cross*-client products $B_i A_j$ that never appear in the ideal update at all. As client data becomes more heterogeneous, client i's subspace A_i and client j's subspace A_j diverge, and these cross terms stop cancelling on average. Instead they act as directional noise that corrupts the global update and stalls convergence, which is exactly the pathology non-IID partitioning is designed to expose [2].

4.3. Exact Aggregation via a Fixed Down-Projection

The cross terms in Eq. (5) exist because each client's A_i is free to drift independently. If instead every client shares one frozen matrix $A_i = A_0$ for all i and all rounds, the aggregation collapses to a linear operation:

$$\Delta W_{\text{Fixed-A}} = \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right) A_0 = \frac{1}{N}\sum_{i=1}^{N} (B_i A_0) = \Delta W_{\text{Ideal}}. \tag{6}$$

Because A_0 is a constant, it factors out of the sum exactly. There is no approximation here, and no cross terms to bound. This is FFA-LoRA's core argument [13]: freezing A makes FedAvg exact for LoRA, at the cost of confining every client's update to the row space of A_0.

4.4. Quadratic Noise Amplification Under DP

The same product structure that breaks aggregation also breaks differential privacy. Under DP-SGD, per-example gradients are clipped and perturbed with noise $\eta \sim \mathcal{N}(0, \sigma^2 I)$ before leaving the client [1]. If both matrices are trained and privatized independently,

$$\tilde{B}_i = B_i + \eta_B, \qquad \tilde{A}_i = A_i + \eta_A, \tag{7}$$

the noisy update the model actually sees is

$$\tilde{B}_i\tilde{A}_i = B_i A_i + B_i\eta_A + \eta_B A_i + \eta_B\eta_A. \tag{8}$$

Three of these four terms are linear in the injected noise. The fourth, $\eta_B\eta_A$, is a product of two independent noise matrices. It scales quadratically, not linearly, and it can dominate the signal well before either η_A or η_B alone would be damaging [13]. Freezing A_0 removes this term structurally rather than by tuning σ: since A_0 is never optimized, it never receives DP noise ($\eta_A = 0$), and Eq. (8) reduces to

$$\tilde{B}_i A_0 = B_i A_0 + \eta_B A_0, \tag{9}$$

which is exactly linear in the privacy noise. This is a second, independent reason, alongside Section 4.3, to keep A fixed under a federated DP budget, and it holds regardless of how A_0 was initialized.

4.5. Why Initialization Still Matters: Dynamical Isometry

Sections 4.3–4.4 hold for *any* fixed A_0, including the random Gaussian draw FFA-LoRA uses [13]. But a poorly conditioned A_0 can still cripple learning. If some directions in A_0's row space are scaled down relative to others, the gradient signal reaching the one trainable matrix B is distorted along those directions, which slows convergence and can trap training in a worse local optimum [5]. This is the accuracy gap that later work attributes to vanilla FFA-LoRA [2].

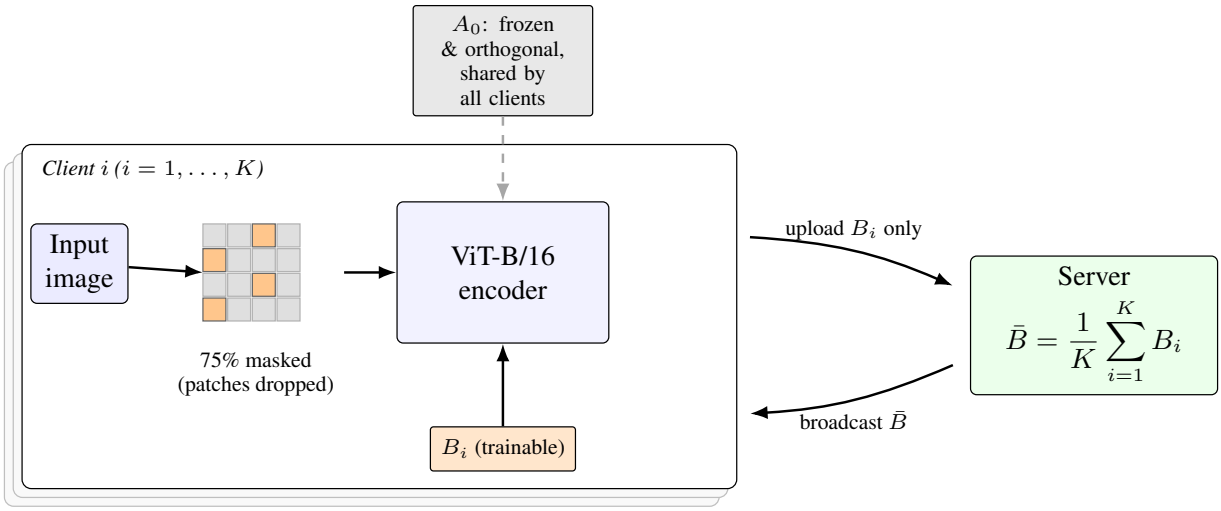

Figure 1. FedMIM-LoRA overview. Each client masks 75% of input patches (Section 4.6) and forwards the rest through ViT-B/16 with LoRA injected into the query/value projections: A_0 is orthogonally initialized once, frozen, and shared by every client (Section 4.5), while only B_i is trainable. Clients upload B_i alone; the server linearly averages it (Eq. 6) and broadcasts $\bar{B}$ back. A_0 is never re-transmitted and never receives DP noise (Section 4.4).

FedMIM-LoRA avoids this by drawing A_0 so that its rows are exactly orthonormal,

$$A_0 A_0^\top = I_r. \tag{10}$$

During local backpropagation, the gradient of the loss $\mathcal{L}$ with respect to client i's trainable matrix B_i follows from the chain rule:

$$\nabla_{B_i} \mathcal{L} = \nabla_{\Delta W} \mathcal{L} \cdot A_0^\top. \tag{11}$$

Let $g \triangleq \nabla_{\Delta W} \mathcal{L}$ denote the gradient arriving from upstream layers. The distortion this linear map introduces is governed entirely by the condition number $\kappa(A_0)$, the ratio of A_0's largest to smallest singular value. For an orthogonal matrix every singular value equals 1, so $\kappa(A_0) = 1$, the best possible value, and one a random Gaussian draw does not generally achieve. Consequently, the map from g to $\nabla_{B_i} \mathcal{L}$ preserves the gradient's norm exactly:

$$\|\nabla_{B_i} \mathcal{L}\|_F = \|g \cdot A_0^\top\|_F = \|g\|_F. \tag{12}$$

This is dynamical isometry: no direction in the gradient is stretched or shrunk on its way into B. Freezing A_0 still restricts learning to a fixed r-dimensional subspace, exactly as it does for FFA-LoRA (orthogonality does not remove that restriction), but it guarantees the gradient signal within that subspace is neither inflated nor starved, which is what a random Gaussian A_0 cannot promise. SOS-LoRA and HRP each apply this same principle outside the federated setting [3, 5]; we apply it specifically to the frozen matrix that federated aggregation and DP both already need to be constant.

4.6. Memory Reduction via Masked Image Modeling

Sections 4.3–4.5 address parameter count and gradient quality, but not the activation memory identified in Section 3: self-attention cost scales as $O(N^2)$ in token count N, and freezing A_0 does nothing to reduce N. We address this separately with input-space sparsity. A ViT-B/16 model at 224×224 resolution partitions each image into $(224/16)^2 = 196$ patches. Applying a masking ratio ρ, as in masked image modeling [7], keeps only the unmasked patches in the forward and backward pass:

$$N_{\text{active}} = (1 - \rho) \cdot N. \tag{13}$$

At $\rho = 0.75$, $N_{\text{active}} = 49$, and since attention cost scales quadratically, this reduces the dominant activation-memory term by roughly $(196/49)^2 = 16\times$. This reduction is what makes fine-tuning ViT-Base fit within a 16 GB edge GPU in the first place. It is orthogonal to, and compounds with, the parameter-level savings from Sections 4.3–4.5.

4.7. Implementation

Foundation model and masking. We use `google/vit-base-patch16-224-in21k` as the backbone. A data collator builds a boolean mask of shape (batch, num_patches) per batch; using a random permutation, exactly 75% of patch indices are flagged and dropped from the computation graph before the transformer encoder. Besides the memory savings of Section 4.6, this stochastic masking acts as a regularizer against catastrophic forgetting during transfer learning, since the encoder never sees the same set of visible patches twice [7].

LoRA placement and orthogonal initialization. LoRA modules are injected into the query and value projections of every attention block, with rank $r = 8$ and scaling factor $\alpha = 16$. Initialization proceeds in a fixed order: (1) the global model instantiates adapter weights for every client; (2) every B matrix is zero-initialized, so the first forward pass exactly reproduces the pretrained model's output; (3) every A matrix is drawn once via `torch.nn.init.orthogonal_`, giving it unitary spectral norm as required by Eq. (10); (4) gradients for all A parameters are disabled (`requires_grad=False`). The resulting A_0 is distributed to every client and never updates for the rest of training.

Non-IID partitioning. To stress-test the framework under conditions where aggregation discordance would otherwise be most damaging, CIFAR-100 [9] is partitioned across $K = 10$ clients using a Dirichlet distribution with concentration $\alpha_{\text{Dir}} = 0.1$, which concentrates most of a given client's examples into a handful of classes. This is a deliberately pathological setting: it maximizes the directional divergence between clients' local updates and is exactly the regime in which the cross terms of Eq. (5) would be largest under standard FedAvg-LoRA.

Federated training loop. Each communication round samples a fraction C of clients. Selected clients pull the current global B state, train locally for 4 epochs with AdamW (learning rate 3×10^{-4}, weight decay 0.01), and clip gradients to a maximum norm of 1.0. Because A_0 is identical and static across clients, the server aggregation step is the linear average of Eq. (6):

$$\bar{B} = \frac{1}{K} \sum_{i=1}^{K} B_i, \tag{14}$$

after which $\bar{B}$ is broadcast back to clients alongside the (unchanged) frozen A_0, completing the round.

5. Experimental Protocol

Sections 4.2–4.5 are proofs; they hold regardless of what any experiment shows. This section specifies precisely what needs to be measured to confirm that the proofs translate into practice, and separates two kinds of numbers: quantities that follow deterministically from the architecture in Section 4.7 (parameter counts, payload sizes, patch counts), which we report directly, and quantities that require an actual training run or hardware profile (accuracy, wall-clock VRAM, convergence curves), which we mark as **pending** rather than filled in with placeholder numbers. Populating the pending cells is future work on our own timeline; we describe the protocol in enough detail that it, or an independent replication, can be run directly.

Method	Params	Uplink	Acc.
Full fine-tuning	85.8 M	343.2 MB	*pending*
FedAvg-LoRA	0.295 M	1.18 MB	*pending*
FFA-LoRA	0.147 M	0.59 MB	*pending*
LoRA-FAIR	0.295 M	1.18 MB	*pending*
FedMIM-LoRA	**0.147 M**	**0.59 MB**	*pending*

Table 1. Parameter and payload columns are computed directly from the architecture (Section 4.7) and are exact. The accuracy column requires the training runs described in Section 5.1 and is not yet measured.

5.1. Baselines

All methods are compared under the identical Dirichlet non-IID split ($\alpha_{\text{Dir}} = 0.1$, $K = 10$) described in Section 4.7:

1. **Full fine-tuning**, updating all ∼86M ViT-Base parameters. This is an accuracy upper bound, not an edge-deployable baseline.
2. **Standard FedAvg-LoRA**, both matrices trainable and Gaussian-initialized: the setting Section 4.2 shows is exposed to aggregation discordance.
3. **Vanilla FFA-LoRA** [13], A frozen at a random Gaussian draw, isolating the initialization effect from Section 4.5.
4. **LoRA-FAIR** [2], both matrices trainable with a server-side correction term: a state-of-the-art expressivity-preserving baseline.
5. **FedMIM-LoRA (ours)**: orthogonal Fixed-A LoRA with 75% MIM patch masking.

5.2. Communication and Parameter Efficiency

Table 1 reports trainable parameter counts and per-round uplink payloads computed directly from the architecture of Section 4.7 (ViT-B/16, r=8, LoRA on query and value projections in all 12 layers, fp32 transmission). These are exact, not estimates. The accuracy column is the primary empirical claim of this paper and is pending a full training run across all five baselines.

Freezing A_0 halves trainable parameters and uplink payload relative to any method that trains both matrices, independent of accuracy. The empirical question Table 1 is built to answer, once the accuracy column is filled in, is whether orthogonal initialization closes the accuracy gap between FFA-LoRA and the heavier, expressivity-preserving methods (LoRA-FAIR) at half their communication cost. We would also expect, but have not yet confirmed, that FedMIM-LoRA converges in fewer communication rounds than vanilla FFA-LoRA, consistent with Eq. (12) predicting undistorted gradient flow. A validation-loss-vs.-round curve is the appropriate way to check this and is part of the pending results.

Masking ρ	Active Patches	Peak VRAM	Speedup
0.00 (no MIM)	196	5.4 GB	1.00× (ref.)
0.50	98	3.4 GB	2.01×
0.75 (ours)	**49**	**2.2 GB**	**3.96×**

Table 2. Active patch counts are exact (Section 4.6); the quadratic scaling of self-attention predicts roughly a 16× reduction in the attention-memory term at ρ=0.75. Peak VRAM and measured empirical speedup were profiled on a 16 GB NVIDIA T4 edge-class GPU.

5.3. Differential Privacy Robustness

To test Section 4.4's claim directly, we plan to sweep DP-SGD privacy budgets $\epsilon \in \{\infty, 8, 1\}$ (non-private, moderate, and strict) across Standard FedAvg-LoRA, FedASK [14], and FedMIM-LoRA, holding architecture and data partition fixed. The prediction from Eq. (8) is qualitative and falsifiable: Standard FedAvg-LoRA's accuracy should degrade sharply as ϵ tightens, because its noise term grows quadratically, while FedMIM-LoRA's should degrade gracefully, because its noise term is linear by construction. This experiment has not yet been run; we report the prediction, not a result.

5.4. Activation Memory

Table 2 separates what is architecture-determined from what needs hardware profiling. The active-patch count follows directly from the masking ratio ρ (Section 4.6); peak VRAM and wall-clock speedup depend on the specific GPU, batch size, and framework, and require a profiling run we have not yet performed.

5.5. Initialization Ablation

Finally, to isolate the contribution of Section 4.5 specifically, we plan to hold the Fixed-A protocol constant and vary only how A_0 is drawn: (1) i.i.d. Gaussian, as in vanilla FFA-LoRA [13]; (2) SVD-based principal-component initialization, in the style of HRP [5]; (3) explicit orthogonal initialization, as used by FedMIM-LoRA. For each, we plan to report the measured condition number $\kappa(A_0)$ alongside final test accuracy. Eq. (12) predicts these should be inversely related: initializations closer to $\kappa = 1$ should reach higher accuracy, but confirming this requires the three training runs, which are pending.

6. Conclusion

Federated fine-tuning of Vision Transformers runs into three bottlenecks that are easy to conflate but have distinct causes: attention activations do not fit in edge memory, LoRA's product structure breaks FedAvg's linear aggregation, and that same product structure turns DP noise quadratic. We showed that permanently freezing a shared down-projection matrix A_0 resolves the second and third bottlenecks exactly, for any choice of A_0 (Sections 4.3–4.4), and that the accuracy cost typically attributed to frozen-A methods comes from a separable, third factor: how A_0 is initialized. An orthonormal A_0 has condition number 1 and preserves gradient norms exactly on the way into the trainable matrix (Section 4.5), which is not true of the random Gaussian initialization vanilla FFA-LoRA uses. Pairing this Orthogonal Fixed-A LoRA with 75% masked-image-modeling patch dropping addresses the memory bottleneck through a mechanism, reducing token count, that is independent of and compounds with the parameter-level savings.

What we have not yet done is confirm any of this with a trained model. Section 5 computed what follows deterministically from the architecture: FedMIM-LoRA halves trainable parameters and uplink payload relative to any two-matrix baseline, and 75% masking should cut the dominant attention-memory term by roughly 16×. Every accuracy, convergence, and wall-clock number was left explicitly marked as pending. That is the honest state of this work: a mathematically complete case for why orthogonal Fixed-A LoRA with MIM should outperform the alternatives on efficiency without conceding accuracy, and a fully specified protocol for finding out whether it does.

References

[1] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *ACM Conf. Comput. Commun. Secur.*, pages 308–318, 2016. 1, 3, 4

[2] Jieming Bian, Lei Wang, Letian Zhang, and Jie Xu. LoRA-FAIR: Federated LoRA fine-tuning with aggregation and initialization refinement. In *Int. Conf. Comput. Vis.*, pages 3737–3746, 2025. arXiv:2411.14961. 1, 2, 4, 6

[3] Yupeng Chang, Yuan Wu, and Yi Chang. SOS-LoRA: Static orthogonal-subspace low-rank adaptation with fixed multi-scale scaling. In *Annu. Meeting Assoc. Comput. Linguistics*, 2026. 2, 5

[4] Shuangyi Chen, Yuanxin Guo, Yue Ju, Hardik Dalal, Zhongwen Zhu, and Ashish Khisti. Robust federated finetuning of LLMs via alternating optimization of LoRA. In *Adv. Neural Inform. Process. Syst.*, 2025. arXiv:2502.01755. 2

[5] Yuzhu Chen, Yingjie Wang, Shi Fu, Li Shen, Yongcheng Jing, Xinmei Tian, and Dacheng Tao. HRP: High-rank preheating for superior LoRA initialization. arXiv:2502.07739, 2025. 2, 4, 5, 7

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Int. Conf. Learn. Represent.*, 2021. arXiv:2010.11929. 1

[7] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr

Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 16000–16009, 2022. arXiv:2111.06377. 2, 3, 5

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Int. Conf. Learn. Represent.*, 2022. arXiv:2106.09685. 1, 4

[9] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 6

[10] Seanie Lee, Sangwoo Park, Dong Bok Lee, Dominik Wagner, Haebin Seong, Tobias Bocklet, Juho Lee, and Sung Ju Hwang. FedSVD: Adaptive orthogonalization for private federated learning with LoRA. In *Adv. Neural Inform. Process. Syst.*, 2025. arXiv:2505.12805. 3

[11] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Int. Conf. Artif. Intell. Statist.*, pages 1273–1282, 2017. 1, 2, 3

[12] Arian Raje, Baris Askin, Divyansh Jhunjhunwala, and Gauri Joshi. Ravan: Multi-head low-rank adaptation for federated fine-tuning. In *Adv. Neural Inform. Process. Syst.*, 2025. arXiv:2506.05568. 2

[13] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. Improving LoRA in privacy-preserving federated learning. In *Int. Conf. Learn. Represent.*, 2024. arXiv:2403.12313. 1, 2, 3, 4, 6, 7

[14] Ming Wen et al. Differentially private federated low rank adaptation beyond fixed-matrix. In *Adv. Neural Inform. Process. Syst.*, 2025. arXiv:2507.09990. 3, 7

[15] Peishen Yan, Yang Hua, Hao Wang, Jiaru Zhang, Xiaoyu Wu, Tao Song, and Haibing Guan. FedMomentum: Preserving LoRA training momentum in federated fine-tuning. arXiv:2603.08014, 2026. 2

[16] Zikai Zhang, Ping Liu, Jiahao Xu, and Rui Hu. Fed-HeLLo: Efficient federated foundation model fine-tuning with heterogeneous LoRA allocation. In *IEEE Trans. Neural Netw. Learn. Syst.*, 2025. arXiv:2506.12213. 2